Návrhové vzory

Creational patterns

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Co jsou návrhové vzory?

Co si představíte pod pojmem **návrhový vzor**?

Co jsou návrhové vzory?

Co si představíte pod pojmem **návrhový vzor**?

Setkali jste se již s nějakým návrhovým vzorem?

Co jsou návrhové vzory?

Návrhové vzory (Design Patterns) jsou

Co jsou návrhové vzory?

Návrhové vzory (Design Patterns) jsou osvědčená řešení běžných problémů v návrhu softwaru

Co jsou návrhové vzory?

Návrhové vzory (Design Patterns) jsou osvědčená řešení běžných problémů v návrhu softwaru

- Představují **šablony** pro řešení opakujících se problémů

Co jsou návrhové vzory?

Návrhové vzory (Design Patterns) jsou osvědčená řešení běžných problémů v návrhu softwaru

- Představují **šablony** pro řešení opakujících se problémů
- Jsou to **abstraktní popisy**, ne konkrétní implementace

Co jsou návrhové vzory?

Návrhové vzory (Design Patterns) jsou osvědčená řešení běžných problémů v návrhu softwaru

- Představují **šablony** pro řešení opakujících se problémů
- Jsou to **abstraktní popisy**, ne konkrétní implementace
- Byly poprvé systematicky popsány v knize Design Patterns: Elements of Reusable Object-Oriented Software (1994)

Co jsou návrhové vzory?

Návrhové vzory (Design Patterns) jsou osvědčená řešení běžných problémů v návrhu softwaru

- Představují **šablony** pro řešení opakujících se problémů
- Jsou to **abstraktní popisy**, ne konkrétní implementace
- Byly poprvé systematicky popsány v knize Design Patterns: Elements of Reusable Object-Oriented Software (1994) („Gang of Four“)

Co jsou návrhové vzory?

Návrhové vzory (Design Patterns) jsou osvědčená řešení běžných problémů v návrhu softwaru

- Představují **šablony** pro řešení opakujících se problémů
- Jsou to **abstraktní popisy**, ne konkrétní implementace
- Byly poprvé systematicky popsány v knize Design Patterns: Elements of Reusable Object-Oriented Software (1994) („Gang of Four“)
- Pomáhají vytvářet **udržovatelný** a **škálovatelný** kód

Co jsou návrhové vzory?

Proč jsou návrhové vzory důležité?

Co jsou návrhové vzory?

Proč jsou návrhové vzory důležité?

- **Znovupoužitelnost** - osvědčená řešení

Co jsou návrhové vzory?

Proč jsou návrhové vzory důležité?

- **Znovupoužitelnost** - osvědčená řešení
- **Komunikace** - společný jazyk mezi vývojáři

Co jsou návrhové vzory?

Proč jsou návrhové vzory důležité?

- **Znovupoužitelnost** - osvědčená řešení
- **Komunikace** - společný jazyk mezi vývojáři
- **Udržovatelnost** - předvídatelná struktura kódu

Co jsou návrhové vzory?

Proč jsou návrhové vzory důležité?

- **Znovupoužitelnost** - osvědčená řešení
- **Komunikace** - společný jazyk mezi vývojáři
- **Udržovatelnost** - předvídatelná struktura kódu
- **Škálovatelnost** - snadné rozšiřování funkcionality

Co jsou návrhové vzory?

Proč jsou návrhové vzory důležité?

- **Znovupoužitelnost** - osvědčená řešení
- **Komunikace** - společný jazyk mezi vývojáři
- **Udržovatelnost** - předvídatelná struktura kódu
- **Škálovatelnost** - snadné rozšiřování funkcionality
- **Testovatelnost** - lepší struktura = snazší testování

Kde se návrhové vzory používají?

Návrhové vzory se používají prakticky všude:

Kde se návrhové vzory používají?

Návrhové vzory se používají prakticky všude:

- **Frameworky a knihovny** - React, Angular, Vue, Express.js, NestJS, ...

Kde se návrhové vzory používají?

Návrhové vzory se používají prakticky všude:

- **Frameworky a knihovny** - React, Angular, Vue, Express.js, NestJS, …
- **Databáze** - ORM jako TypeORM, Prisma, …

Kde se návrhové vzory používají?

Návrhové vzory se používají prakticky všude:

- **Frameworky a knihovny** - React, Angular, Vue, Express.js, NestJS, ...
- **Databáze** - ORM jako TypeORM, Prisma, ...
- **State management** - Redux, MobX

Kde se návrhové vzory používají?

Návrhové vzory se používají prakticky všude:

- **Frameworky a knihovny** - React, Angular, Vue, Express.js, NestJS, ...
- **Databáze** - ORM jako TypeORM, Prisma, ...
- **State management** - Redux, MobX
- **DOM manipulace** - jQuery internals, Alpine.js, ...

Kde se návrhové vzory používají?

Příklad z reálného světa:

Kde se návrhové vzory používají?

Příklad z reálného světa:

```
// React uses Observer pattern for state management
const [count, setCount] = useState(0);
```

Kde se návrhové vzory používají?

Příklad z reálného světa:

```
// React uses Observer pattern for state management
const [count, setCount] = useState(0);

// Express.js uses Middleware pattern
app.use(express.json());
```

Kde se návrhové vzory používají?

Příklad z reálného světa:

```
// React uses Observer pattern for state management
const [count, setCount] = useState(0);

// Express.js uses Middleware pattern
app.use(express.json());

// TypeORM uses Repository pattern
const userRepository = dataSource.getRepository(User);
```

Kategorie návrhových vzorů

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů
   - Factory, Builder, Singleton

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů
   - Factory, Builder, Singleton
2. **Structural** (Strukturální)

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů
   - Factory, Builder, Singleton
2. **Structural** (Strukturální)
   - Řeší kompozici tříd a objektů

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů
   - Factory, Builder, Singleton
2. **Structural** (Strukturální)
   - Řeší kompozici tříd a objektů
   - Adapter, Decorator, Composite

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů
   - Factory, Builder, Singleton
2. **Structural** (Strukturální)
   - Řeší kompozici tříd a objektů
   - Adapter, Decorator, Composite
3. **Behavioral** (Behaviorální)

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů
   - Factory, Builder, Singleton
2. **Structural** (Strukturální)
   - Řeší kompozici tříd a objektů
   - Adapter, Decorator, Composite
3. **Behavioral** (Behaviorální)
   - Řeší komunikaci mezi objekty

Kategorie návrhových vzorů

1. **Creational** (Vytvářející)
   - Řeší vytváření objektů
   - Factory, Builder, Singleton
2. **Structural** (Strukturální)
   - Řeší kompozici tříd a objektů
   - Adapter, Decorator, Composite
3. **Behavioral** (Behaviorální)
   - Řeší komunikaci mezi objekty
   - Observer, Iterator, Strategy

Creational Patterns

Factory Pattern

Factory Pattern

Jaký problém řeší **Factory Pattern**?

Factory Pattern

Jaký problém řeší **Factory Pattern**?

Jak byste vytvořili objekty různých typů bez znalosti jejich konkrétních tříd?

Factory Pattern

Představte si **výrobní linku v továrně**

Factory Pattern

Představte si **výrobní linku v továrně**

- Zákazník si objedná produkt (např. „červené auto“)

Factory Pattern

Představte si **výrobní linku v továrně**

- Zákazník si objedná produkt (např. „červené auto“)
- Továrna rozhodne, kterou linku použít

Factory Pattern

Představte si **výrobní linku v továrně**

- Zákazník si objedná produkt (např. „červené auto“)
- Továrna rozhodne, kterou linku použít
- Zákazník dostane hotový produkt, aniž by znal detaily výroby

Factory Pattern

Představte si **výrobní linku v továrně**

- Zákazník si objedná produkt (např. „červené auto“)
- Továrna rozhodne, kterou linku použít
- Zákazník dostane hotový produkt, aniž by znal detaily výroby

Factory Pattern funguje stejně - **skrývá složitost vytváření objektů**

Factory Pattern

Factory Pattern (Tovární vzor)

Factory Pattern

Factory Pattern (Tovární vzor) umožňuje vytvářet objekty bez specifikace jejich přesné třídy

Factory Pattern

Factory Pattern (Tovární vzor) umožňuje vytvářet objekty bez specifikace jejich přesné třídy

- Místo přímého volání konstruktoru používáme **factory metodu**

Factory Pattern

Factory Pattern (Tovární vzor) umožňuje vytvářet objekty bez specifikace jejich přesné třídy

- Místo přímého volání konstruktoru používáme **factory metodu**
- Factory rozhoduje, jakou třídu instancovat

Factory Pattern

Factory Pattern (Tovární vzor) umožňuje vytvářet objekty bez specifikace jejich přesné třídy

- Místo přímého volání konstruktoru používáme **factory metodu**
- Factory rozhoduje, jakou třídu instancovat
- Umožňuje **flexibilní** vytváření objektů

Factory Pattern

Factory Pattern (Tovární vzor) umožňuje vytvářet objekty bez specifikace jejich přesné třídy

- Místo přímého volání konstruktoru používáme **factory metodu**
- Factory rozhoduje, jakou třídu instancovat
- Umožňuje **flexibilní** vytváření objektů
- **Návratový typ** factory metody **je rozhraní (interface)**

Factory Pattern

Kdy použít Factory Pattern?

Factory Pattern

Kdy použít Factory Pattern?

- Když nevíme předem, jakou třídu budeme potřebovat

Factory Pattern

Kdy použít Factory Pattern?

- Když nevíme předem, jakou třídu budeme potřebovat
- Když chceme **centralizovat** logiku vytváření objektů

Factory Pattern

Kdy použít Factory Pattern?

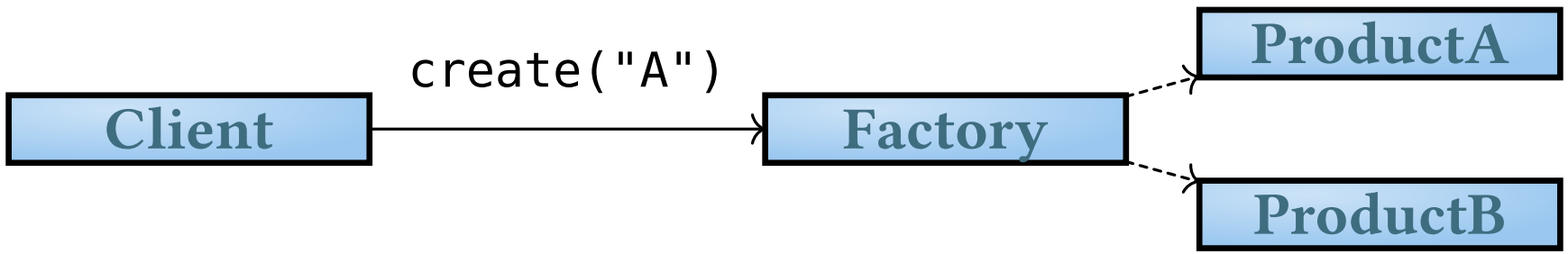
- Když nevíme předem, jakou třídu budeme potřebovat
- Když chceme **centralizovat** logiku vytváření objektů
- Když chceme **skrýt** složitost vytváření objektů

Factory Pattern

Kdy použít Factory Pattern?

- Když nevíme předem, jakou třídu budeme potřebovat
- Když chceme **centralizovat** logiku vytváření objektů
- Když chceme **skrýt** složitost vytváření objektů
- Když potřebujeme podporovat **různé typy** objektů

Struktura Factory Pattern



Příklad Factory

```
interface Animal {
  makeSound(): void;
}

class Dog implements Animal {
  makeSound(): void { console.log("Woof!"); }
}

class Cat implements Animal {
  makeSound(): void { console.log("Meow!"); }
}
```

Příklad Factory

```
class AnimalFactory {
  static createAnimal(type: string): Animal {
    switch (type) {
      case "dog": return new Dog();
      case "cat": return new Cat();
      default:
        throw new Error(`Unknown animal type: ${type}`);
    }
  }
}
```

Příklad Factory

```
const dog = AnimalFactory.createAnimal("dog");
dog.makeSound(); // "Woof!"
```

Pokročilejší příklad Factory

```
type AnimalConstructor = new () => Animal;

class AnimalFactory {
  private static creators:
    Map<string, AnimalConstructor> = new Map();
...
```

Pokročilejší příklad Factory

```
class AnimalFactory {
  ...
  static register(
    type: string,
    creator: AnimalConstructor
  ): void {
    this.creators.set(type, creator);
  }
  ...
```

Pokročilejší příklad Factory

```
class AnimalFactory {
  ...
  static create(type: string): Animal {
    const Creator = this.creators.get(type);
    if (!Creator) {
      throw new Error(`Unknown type: ${type}`);
    }

    return new Creator();
  }
}
```

Pokročilejší příklad Factory

```
// Register types
AnimalFactory.register("dog", Dog);
AnimalFactory.register("cat", Cat);

// Usage
const animal = AnimalFactory.create("dog");
animal.makeSound(); // "Woof!"
```

Pokročilejší příklad Factory

Výhody tohoto přístupu:

Pokročilejší příklad Factory

Výhody tohoto přístupu:

- Snadné přidávání nových typů

Pokročilejší příklad Factory

Výhody tohoto přístupu:

- Snadné přidávání nových typů
- Decentralizovaná registrace

Pokročilejší příklad Factory

Výhody tohoto přístupu:

- Snadné přidávání nových typů
- Decentralizovaná registrace
- Flexibilní konfigurace

Builder Pattern

Builder Pattern

Jak byste vytvořili objekt s **10 volitelnými parametry**?

Builder Pattern

Jak byste vytvořili objekt s **10 volitelnými parametry**?

Byl by takový konstruktor čitelný a snadno použitelný?

Builder Pattern

Představte si **stavbu domu**

Builder Pattern

Představte si **stavbu domu**

- Nejprve základy, pak zdi, střecha, okna, …

Builder Pattern

Představte si **stavbu domu**

- Nejprve základy, pak zdi, střecha, okna, …
- Můžete si vybrat různé materiály a styly

Builder Pattern

Představte si **stavbu domu**

- Nejprve základy, pak zdi, střecha, okna, …
- Můžete si vybrat různé materiály a styly
- Na konci dostanete hotový dům

Builder Pattern

Představte si **stavbu domu**

- Nejprve základy, pak zdi, střecha, okna, …
- Můžete si vybrat různé materiály a styly
- Na konci dostanete hotový dům

Builder Pattern staví objekty **krok za krokem**

Builder Pattern

Builder Pattern (Stavitel)

Builder Pattern

Builder Pattern (Stavitel) umožňuje vytvářet složité objekty krok za krokem

Builder Pattern

Builder Pattern (Stavitel) umožňuje vytvářet složité objekty krok za krokem

- Odděluje konstrukci objektu od jeho reprezentace

Builder Pattern

Builder Pattern (Stavitel) umožňuje vytvářet složité objekty krok za krokem

- Odděluje konstrukci objektu od jeho reprezentace
- Umožňuje vytvářet různé varianty objektu

Builder Pattern

Builder Pattern (Stavitel) umožňuje vytvářet složité objekty krok za krokem

- Odděluje konstrukci objektu od jeho reprezentace
- Umožňuje vytvářet různé varianty objektu
- Zlepšuje čitelnost kódu při vytváření složitých objektů

Builder Pattern

Kdy použít Builder Pattern?

Builder Pattern

Kdy použít Builder Pattern?

- Když má objekt **mnoho parametrů** konstruktoru

Builder Pattern

Kdy použít Builder Pattern?

- Když má objekt **mnoho parametrů** konstruktoru
- Když chceme vytvářet objekty **postupně**

Builder Pattern

Kdy použít Builder Pattern?

- Když má objekt **mnoho parametrů** konstruktoru
- Když chceme vytvářet objekty **postupně**
- Když potřebujeme různé **varianty** stejného objektu

Builder Pattern

Kdy použít Builder Pattern?

- Když má objekt **mnoho parametrů** konstruktoru
- Když chceme vytvářet objekty **postupně**
- Když potřebujeme různé **varianty** stejného objektu
- Když chceme **validovat** parametry před vytvořením objektu

Builder Pattern

Příklad - Builder pro User:

```
class User {
  constructor(
    public name: string,
    public email: string,
    public age?: number,
    public phone?: string,
    public address?: string,
    public roles: string[] = []
  ) {}
}
```

Builder Pattern

Řešení pomocí Builder Pattern:

```
class UserBuilder {
  private name!: string;
  private email!: string;
  private age?: number;
  private phone?: string;
  private address?: string;
  private roles: string[] = [];
  ...
```

Builder Pattern

```
...
setName(name: string): this {
  this.name = name;
  return this;
}

setEmail(email: string): this {
  this.email = email;
  return this;
}
```

Builder Pattern

```
  setAge(age: number): this {
    this.age = age;
    return this;
  }

  setPhone(phone: string): this {
    this.phone = phone;
    return this;
  }
  ...
```

Builder Pattern

```
...
setAddress(address: string): this {
  this.address = address;
  return this;
}

addRole(role: string): this {
  this.roles.push(role);
  return this;
}
```

Builder Pattern

```
    ...
    build(): User {
      if (!this.name || !this.email) {
        throw new Error("Name and email are required");
      }
      return new User(
        this.name, this.email, this.age,
        this.phone, this.address, this.roles
      );
    }
}
```

Builder Pattern

Použití Builder Pattern:

```
const user = new UserBuilder()
  .setName("John Doe")
  .setEmail("john@example.com")
  .setAge(30)
  .setPhone("123-456-7890")
  .setAddress("123 Main St")
  .build();
```

Builder Pattern

Výhody:

Builder Pattern

Výhody:

- **Čitelný** kód

Builder Pattern

Výhody:

- **Čitelný** kód
- **Flexibilní** vytváření

Builder Pattern

Výhody:

- **Čitelný** kód
- **Flexibilní** vytváření
- **Validace** před vytvořením objektu

Builder Pattern

Výhody:

- **Čitelný** kód
- **Flexibilní** vytváření
- **Validace** před vytvořením objektu
- **Fluent interface** (method chaining)

Builder Pattern

Nevýhody:

Builder Pattern

Nevýhody:

- **Více kódu** - musíte napsat builder třídu

Builder Pattern

Nevýhody:

- **Více kódu** - musíte napsat builder třídu
- **Komplexnost** - pro jednoduché objekty je zbytečný

Builder Pattern

Nevýhody:

- **Více kódu** - musíte napsat builder třídu
- **Komplexnost** - pro jednoduché objekty je zbytečný
- **Nekonzistence** - objekt může být v neplatném stavu během stavby

Builder Pattern

Nevýhody:

- **Více kódu** - musíte napsat builder třídu
- **Komplexnost** - pro jednoduché objekty je zbytečný
- **Nekonzistence** - objekt může být v neplatném stavu během stavby
- **Sdílení reference** - stejná builder instance může být omylem použita vícekrát

Builder Pattern

Pozor na sdílení builder instance!

```
const builder = new UserBuilder()
  .setName("John")
  .setEmail("john@example.com")
  .addRole("user");

const user1 = builder.setAge(25).addRole("admin").build();
const user2 = builder.setAge(30).build();

console.log(user1);
console.log(user2);
```

Builder Pattern

Problém:

Builder Pattern

Problém: Stejná builder instance může být omylem použita vícekrát a nastavit parametry i pro jiné objekty

Builder Pattern

Problém: Stejná builder instance může být omylem použita vícekrát a nastavit parametry i pro jiné objekty

Řešení 1:

Builder Pattern

Problém: Stejná builder instance může být omylem použita vícekrát a nastavit parametry i pro jiné objekty

Řešení 1: Při každé modifikaci builderu vytvořte nový builder - implementujte **immutable builder**

Builder Pattern

Problém: Stejná builder instance může být omylem použita vícekrát a nastavit parametry i pro jiné objekty

Řešení 1: Při každé modifikaci builderu vytvořte nový builder - implementujte **immutable builder**

Řešení 2:

Builder Pattern

Problém: Stejná builder instance může být omylem použita vícekrát a nastavit parametry i pro jiné objekty

Řešení 1: Při každé modifikaci builderu vytvořte nový builder - implementujte **immutable builder**

Řešení 2: Vytvořte si Factory na Builder

```
const user1 = UserBuilderFactory.create(builder) // copy
  .setAge(25).addRole("admin").build();
const user2 = UserBuilderFactory.create(builder) // copy
  .setAge(30).build();
```

Singleton Pattern

Singleton Pattern

Kdy je potřeba mít v aplikaci **pouze jednu instanci** nějaké třídy?

Singleton Pattern

Kdy je potřeba mít v aplikaci **pouze jednu instanci** nějaké třídy?

Jak byste zajistili, že se vytvoří právě jedna instance?

Singleton Pattern

Kdy je potřeba mít v aplikaci **pouze jednu instanci** nějaké třídy?

Jak byste zajistili, že se vytvoří právě jedna instance?

- Zakázat přímé vytváření objektů (`new`)?

Singleton Pattern

Kdy je potřeba mít v aplikaci **pouze jednu instanci** nějaké třídy?

Jak byste zajistili, že se vytvoří právě jedna instance?

- Zakázat přímé vytváření objektů (`new`)?
- Uložit si tu jednu instanci

Singleton Pattern

Kdy je potřeba mít v aplikaci **pouze jednu instanci** nějaké třídy?

Jak byste zajistili, že se vytvoří právě jedna instance?

- Zakázat přímé vytváření objektů (`new`)?
- Uložit si tu jednu instanci
- Kontrolovat, jestli už instance existuje

Singleton Pattern

Představte si **prezidenta státu**

Singleton Pattern

Představte si **prezidenta státu**

- V jednom okamžiku může být jen jeden prezident

Singleton Pattern

Představte si **prezidenta státu**

- V jednom okamžiku může být jen jeden prezident
- Všichni občané přistupují ke stejnému prezidentovi

Singleton Pattern

Představte si **prezidenta státu**

- V jednom okamžiku může být jen jeden prezident
- Všichni občané přistupují ke stejnému prezidentovi
- Nelze vytvořit druhého prezidenta

Singleton Pattern

Představte si **prezidenta státu**

- V jednom okamžiku může být jen jeden prezident
- Všichni občané přistupují ke stejnému prezidentovi
- Nelze vytvořit druhého prezidenta

Singleton zajišťuje **právě jednu instanci** v celé aplikaci

Singleton Pattern

Singleton Pattern (Jedináček) zajišťuje, že třída má pouze **jednu instanci**

Singleton Pattern

Singleton Pattern (Jedináček) zajišťuje, že třída má pouze **jednu instanci**

- Poskytuje **globální přístupový bod** k této instanci

Singleton Pattern

Singleton Pattern (Jedináček) zajišťuje, že třída má pouze **jednu instanci**

- Poskytuje **globální přístupový bod** k této instanci
- **Kontroluje vytváření** instancí (private constructor)

Singleton Pattern

Singleton Pattern (Jedináček) zajišťuje, že třída má pouze **jednu instanci**

- Poskytuje **globální přístupový bod** k této instanci
- **Kontroluje vytváření** instancí (private constructor)
- Užitečné pro **sdílené zdroje**: databáze, loggery, cache

Singleton Pattern

Singleton Pattern (Jedináček) zajišťuje, že třída má pouze **jednu instanci**

- Poskytuje **globální přístupový bod** k této instanci
- **Kontroluje vytváření** instancí (private constructor)
- Užitečné pro **sdílené zdroje**: databáze, loggery, cache
- Když potřebujeme **globální stav** v celé aplikaci

Singleton Pattern

Kdy NEpoužívat Singleton Pattern?

Singleton Pattern

Kdy NEpoužívat Singleton Pattern?

- Když potřebujete **testovatelný** kód - Singleton ztěžuje **mockování**

Singleton Pattern

Kdy NEpoužívat Singleton Pattern?

- Když potřebujete **testovatelný** kód - Singleton ztěžuje **mockování**
- Když chcete **volné propojení** (**loose coupling**) mezi třídami

Singleton Pattern

Kdy NEpoužívat Singleton Pattern?

- Když potřebujete **testovatelný** kód - Singleton ztěžuje **mockování**
- Když chcete **volné propojení** (**loose coupling**) mezi třídami
  - Loose coupling znamená, že třídy nejsou závislé na sobě a mohou být snadno vyměňovány

Singleton Pattern

Kdy NEpoužívat Singleton Pattern?

- Když potřebujete **testovatelný** kód - Singleton ztěžuje **mockování**
- Když chcete **volné propojení** (**loose coupling**) mezi třídami
  - Loose coupling znamená, že třídy nejsou závislé na sobě a mohou být snadno vyměňovány
- Když globální stav může způsobit **nepředvídatelné chování**

Singleton Pattern

Kdy NEpoužívat Singleton Pattern?

- Když potřebujete **testovatelný** kód - Singleton ztěžuje **mockování**
- Když chcete **volné propojení** (**loose coupling**) mezi třídami
  - Loose coupling znamená, že třídy nejsou závislé na sobě a mohou být snadno vyměňovány
- Když globální stav může způsobit **nepředvídatelné chování**
- Když plánujete **paralelní zpracování** (více instancí)

Singleton Pattern

Kdy NEpoužívat Singleton Pattern?

- Když potřebujete **testovatelný** kód - Singleton ztěžuje **mockování**
- Když chcete **volné propojení** (**loose coupling**) mezi třídami
  - Loose coupling znamená, že třídy nejsou závislé na sobě a mohou být snadno vyměňovány
- Když globální stav může způsobit **nepředvídatelné chování**
- Když plánujete **paralelní zpracování** (více instancí)

Alternativa: Dependency Injection - instance se předává jako závislost

Singleton Pattern

Implementace Singleton v TypeScriptu:

```
class Database {
  private static instance: Database;

  private constructor() {
    // Private constructor prevents direct instantiation
    console.log("Database connection established");
  }
```

Singleton Pattern

```
  static getInstance(): Database {
    if (!Database.instance) {
      Database.instance = new Database();
    }
    return Database.instance;
  }
```

Singleton Pattern

```
  query(sql: string): void {
    if (!Database.instance) {
      throw new Error("Database not initialized");
    }
    console.log(`Executing: ${sql}`);
  }
}
```

Singleton Pattern

Použití Singleton:

```
// Always get the same instance
const db1 = Database.getInstance();
const db2 = Database.getInstance();

console.log(db1 === db2); // true

db1.query("SELECT * FROM users");
db2.query("SELECT * FROM posts");
```

Singleton Pattern

Použití Singleton:

```
// Always get the same instance
const db1 = Database.getInstance();
const db2 = Database.getInstance();

console.log(db1 === db2); // true

db1.query("SELECT * FROM users");
db2.query("SELECT * FROM posts");
```

Poznámka: Kvůli **private constructoru** nelze volat `new Database()`

Singleton Pattern

Poznámka k thread-safety

Singleton Pattern

Poznámka k thread-safety

JavaScript/TypeScript běží v **jednom vlákně** (single-threaded)

Singleton Pattern

Poznámka k thread-safety

JavaScript/TypeScript běží v **jednom vlákně** (single-threaded)

- Náš jednoduchý Singleton je **dostatečný** pro JS/TS

Singleton Pattern

Poznámka k thread-safety

JavaScript/TypeScript běží v **jednom vlákně** (single-threaded)

- Náš jednoduchý Singleton je **dostatečný** pro JS/TS
- Thread-safe implementace je potřeba v jiných jazycích (Java, C#,...)

Singleton Pattern

Poznámka k thread-safety

JavaScript/TypeScript běží v **jednom vlákně** (single-threaded)

- Náš jednoduchý Singleton je **dostatečný** pro JS/TS
- Thread-safe implementace je potřeba v jiných jazycích (Java, C#,...)
- V Node.js jsou moduly **cacheovány** a jedná se tedy o přirozený Singleton

Singleton Pattern

Poznámka k thread-safety

JavaScript/TypeScript běží v **jednom vlákně** (single-threaded)

- Náš jednoduchý Singleton je **dostatečný** pro JS/TS
- Thread-safe implementace je potřeba v jiných jazycích (Java, C#,...)
- V Node.js jsou moduly **cacheovány** a jedná se tedy o přirozený Singleton

```
// In Node.js module itself is a Singleton
export const database = new Database();
```

Shrnutí Creational Patterns

Co jsme se naučili o Creational Patterns?

Shrnutí Creational Patterns

Co jsme se naučili o Creational Patterns?

Factory	Centralizované vytváření objektů různých typů
Builder	Postupné vytváření složitých objektů s fluent API
Singleton	Zajištění pouze jedné instance v celé aplikaci

Shrnutí Creational Patterns

Co jsme se naučili o Creational Patterns?

Factory	Centralizované vytváření objektů různých typů
Builder	Postupné vytváření složitých objektů s fluent API
Singleton	Zajištění pouze jedné instance v celé aplikaci

Kdy použít: Když potřebujete kontrolovat **jak** se objekty vytvářejí

Shrnutí Creational Patterns

Co jsme se naučili o Creational Patterns?

Factory	Centralizované vytváření objektů různých typů
Builder	Postupné vytváření složitých objektů s fluent API
Singleton	Zajištění pouze jedné instance v celé aplikaci

Kdy použít: Když potřebujete kontrolovat **jak** se objekty vytvářejí

Existují i další Creational Patterns a je vhodné si je ve volném čase prohlédnout

Shrnutí Creational Patterns

Příště budeme pokračovat: **structural** vzory (Adapter, Decorator, Composite) a **behavioral** vzory (Observer, Iterator, Strategy)